

Tesseract-BP: High-Performance Belief Propagation & Layered Serial SIMD Vectorization

The Tiered QEC Decoding Vision & High-Throughput Min-Sum Architecture

Tesseract QEC Decoding Team

Advanced Agentic Coding / Quantum Decoding Group

August 8, 2026

Lecture Overview

- 1 The Tesseract Vision & QEC Decoding Fundamentals
- 2 qLDPC Codes & Belief Propagation Foundations
- 3 Tesseract-BP Architecture & Optimization Deep-Dive
- 4 Benchmarks, Empirical Results & Future Outlook

The Grand Vision: Tesseract as a One-Stop QEC Engine

Repository Objective

Make **Tesseract** the unified, highly optimized **one-stop platform for all offline Quantum Error Correction (QEC) decoding needs**.

Tier 1: Fast Baseline

Belief Propagation (BP) + OSD

Fast heuristic evaluation ($10^5 - 10^6$ shots/s). First stop for code construction & rapid feedback.

Tier 2: Refined Accuracy

MultiPass / Gari Tesseract

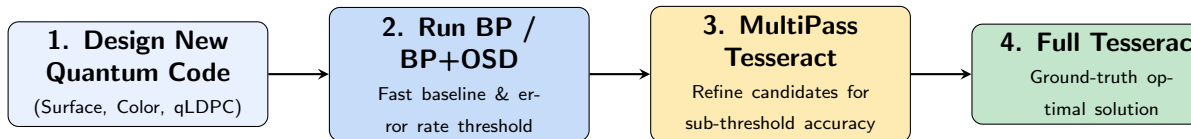
Heuristic path-search refinement for near-threshold verification.

Tier 3: Exact Optimum

Full Tesseract (A* Search)

Exact optimal path decoding for ground-truth physical error rate bounds.

Recommended Code Development Workflow



Why start with BP?

- Ultra-high throughput enables sampling millions of Monte Carlo shots in seconds.
- Direct measurement of syndrome graph connectivity and trapping set vulnerabilities.

Quantum Error Correction & Syndrome Decoding

Physical Error to Syndrome Mapping

- Physical errors $e \in \mathbb{F}_2^N$ occur on physical qubits.
- Quantum non-demolition (QND) stabilizer measurements yield syndrome $s \in \mathbb{F}_2^M$:

$$s = H \cdot e \pmod{2}$$

- **Parity-Check Matrix (H):** $M \times N$ binary matrix mapping N potential error channels to M detectors.

The Decoding Problem

Given observed syndrome s , find the most likely error candidate $\hat{e}$ satisfying:

$$H \cdot \hat{e} = s \pmod{2}$$

- **Maximum Likelihood Decoding (MLD):**

$$\hat{e}_{MLD} = \arg \max_{e: He=s} P(e)$$

- **NP-Hardness:** MLD over arbitrary linear block codes is strictly NP-hard!

Syndrome Generation from Stim DEM

Detector Error Model (DEM)

- Stim describes circuit-level noise (T1/T2, gate faults) as probabilistic error mechanisms.
- Each error mechanism flips a specific set of parity check measurements (detectors).

Constructing the Parity Check Matrix (H)

- **Columns (N):** Correspond to independent error mechanisms in the DEM.
- **Rows (M):** Correspond to detectors (syndrome bits) triggered by errors.
- $H_{i,j} = 1$ if error j triggers detector i , otherwise 0.
- **Priors:** Each column j has a prior probability p_j defined by the DEM.

The Shift to qLDPC Codes & Tanner Graphs

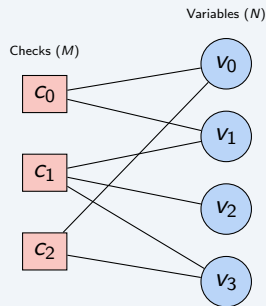
Quantum Low-Density Parity-Check (qLDPC)

- Standard Surface Codes scale as $O(d^2)$ physical qubits per logical qubit.
- **qLDPC Codes** (e.g., Bivariate Bicycle Codes $[[72, 12, 6]]$, $[[144, 12, 12]]$) achieve constant encoding rate:

$$\frac{K}{N} = \text{const} > 0$$

- Parity-check matrix H is sparse (constant row/column weights).

Bipartite Tanner Graph



Brief History of Belief Propagation

- **1962 — Robert Gallager:** Introduces Low-Density Parity-Check (LDPC) codes and iterative belief propagation decoding in his MIT PhD thesis. (Ignored for decades due to hardware limitations!).
- **1982 — Judea Pearl:** Formalizes Belief Propagation on Bayesian networks and Directed Acyclic Graphs (DAGs).
- **1996 — David MacKay & Radford Neal:** Rediscover LDPC codes and demonstrate near Shannon-limit capacity performance over noisy channels.
- **2000s-Present — Quantum Adaptation:** Adaptation of BP and Min-Sum message-passing algorithms to Quantum LDPC and surface codes for real-time and offline decoding.

Log-Likelihood Ratio (LLR) Mathematics

LLR Definition

For a binary random variable $v_i \in \{0, 1\}$, the Log-Likelihood Ratio is defined as:

$$L(v_i) = \ln \left(\frac{P(v_i = 0)}{P(v_i = 1)} \right)$$

- $L(v_i) > 0 \implies v_i = 0$ is more likely (no error).
- $L(v_i) < 0 \implies v_i = 1$ is more likely (error present).
- $|L(v_i)|$ represents the **reliability** / confidence of the belief.

Channel Prior Initialization from Stim DEM

Given physical error probability p_i from the Stim Detector Error Model:

$$L_{\text{prior}}(v_i) = \ln \left(\frac{1 - p_i}{p_i} \right)$$

For integer SIMD execution, LLRs are scaled and fixed into 32-bit signed integers (LLR_INT).

Sum-Product vs. Min-Sum Update Rules

Sum-Product Check Update (Exact)

Check-to-variable message $R_{c \rightarrow v}$ in exact Sum-Product:

$$\tanh\left(\frac{R_{c \rightarrow v}}{2}\right) = \left(\prod_{v' \in N(c) \setminus \{v\}} \text{sign}(Q_{v' \rightarrow c}) \right) \times \prod_{v' \in N(c) \setminus \{v\}} \tanh\left(\frac{|Q_{v' \rightarrow c}|}{2}\right)$$

Requires expensive transcendental evaluations ($\tanh$, atanh) per edge per iteration!

Min-Sum Approximation (Fast Vectorizable)

Applies the identity $\tanh(x) \approx \min(x)$ for large LLRs:

$$R_{c \rightarrow v} = \alpha \times \left(\prod_{v' \in N(c) \setminus \{v\}} \text{sign}(Q_{v' \rightarrow c}) \right) \times \min_{v' \in N(c) \setminus \{v\}} |Q_{v' \rightarrow c}|$$

- **Normalization / Damping Factor:** $\alpha = 0.625$ compensates for Min-Sum over-estimation.

BP Scheduling Modalities: Parallel vs. Serial

1. Synchronous (Parallel) Schedule

Two distinct phases per iteration:

- 1 Compute all $V \rightarrow C$ messages concurrently.
- 2 Compute all $C \rightarrow V$ messages concurrently.

Pros: Massive potential parallelism.

Cons: Delayed message propagation; requires 2x memory iteration state.

2. Asynchronous (Serial) Schedule

Sequential update across nodes:

- 1 Select node (variable or check).
- 2 Immediately update posterior belief L_v .
- 3 Immediately propagate updated belief to neighboring nodes.

Pros: 2x faster iteration convergence!

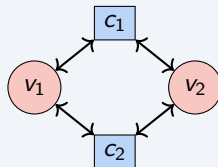
Cons: Requires careful ordering to prevent L1 cache thrashing.

LDPC Trapping Sets & Convergence Obstacles

What is an LDPC Trapping Set?

- A sub-graph of variable nodes and check nodes that fails to converge due to symmetric graph loops.
- In QEC codes (Color codes, Bivariate Bicycle codes), short 4-cycles and 6-cycles form **symmetric trapping sets**.
- Under fixed linear serial schedules ($0 \rightarrow N - 1$), beliefs bounce back and forth between states endlessly.

Trapping Set Traversal



Symmetric 4-Cycle Oscillation

Why Standard BP Struggles with Quantum Codes

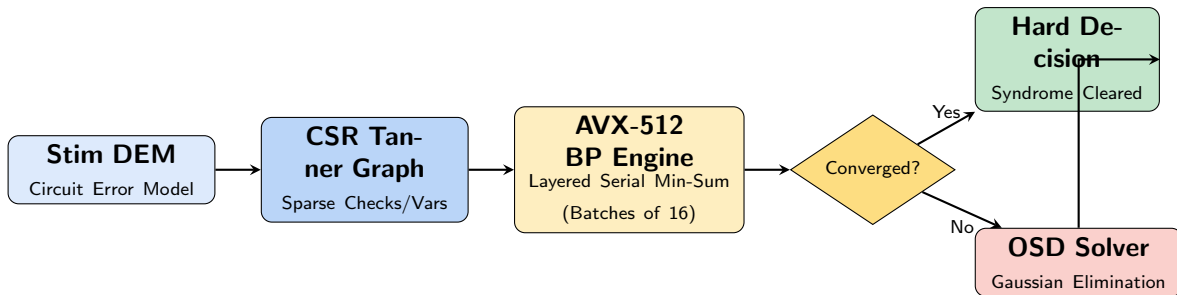
The Quantum Trapping Set Problem

- Classical LDPC codes are designed with large girth (no short cycles).
- Quantum LDPC codes (due to commutativity constraints $H_X H_Z^T = 0$) inherently contain abundant short cycles (length 4 and 6).
- These form **symmetric trapping sets**—sub-graphs that trick BP into endless oscillation.

The Need for OSD and Advanced Scheduling

- **Without Scheduling:** Parallel updates bounce messages back and forth synchronously, failing to break symmetry.
- **Without OSD:** BP gives up when it can't converge, leaving the syndrome uncleared (logical failure).
- **Solution:** We must break graph symmetry (via Stochastic Scheduling) and provide an algebraic fallback (via OSD) when heuristic BP fails.

Tesseract-BP High-Level Data Flow



Architecture: Pointer-Chasing vs. Bidirectional CSR

Naive BP: Pointer-Chasing Graph

```
struct Node {  
    std::vector<Node*> neighbors;  
    std::vector<double> messages;  
};  
std::vector<Node*> nodes;
```

- Follows `std::vector` memory allocations.
- Nodes scattered across fragmented heap memory.
- Traversing neighbors means chasing pointers
→ **L1/L2 Cache Miss Penalty!**

Tesseract BP: Bidirectional CSR

```
1 // Flat integer arrays, no pointers!  
2 vector<int> check_to_var_indices;  
3 vector<int> check_to_var_offsets;  
4 vector<int> var_to_check_indices;  
5 vector<int> var_to_check_offsets;
```

- **Compressed Sparse Row (CSR)**: Graph represented as 1D contiguous arrays.
- **Bidirectional**: Fast $O(1)$ neighborhood lookups in both directions ($C \rightarrow V$ and $V \rightarrow C$).
- Sequential memory access ensures hardware prefetcher efficiency.

Optimization 1: Memory Foundation Rewrite (Interleaved 1D Array)

BEFORE: Vector of Vectors (Non-Interleaved)

```
Shot 0: [ Var0, Var1, Var2, ..., VarN ]  
Shot 1: [ Var0, Var1, Var2, ..., VarN ] (RAM Gap)  
Shot 2: [ Var0, Var1, Var2, ..., VarN ] (RAM Gap)  
...  
Shot 15: [ Var0, Var1, Var2, ..., VarN ]
```

- Loading Variable V_i across 16 SIMD shots required fetching **16 separate RAM addresses**.
- Transfers 1,024 bytes to extract 64 bytes of data → **L1 Cache Flushed!**

AFTER: Flat Interleaved 1D Array

```
RAM: [ Var0_S0..S15 | Var1_S0..S15 | Var2_S0..S15 ]
```

- Memory address: `posteriors_flat[v * 16 + b]`.
- All 16 shot streams for Variable V_i sit **physically contiguous** in RAM (64 bytes).
- **1 SIMD Aligned Load** = 1 L1 Cache Line Hit!

Optimization 2: Horizontal Layered Serial Scheduling

Algorithmic Transformation

Instead of updating variable node posteriors sequentially, iteration loops operate directly over **check nodes** ($c \in C$):

- 1 For each check node c , calculate check-to-variable updates dynamically:

$$Q_{c,v} = L_v - R_{c,v}$$

- 2 Compute the updated check message $R'_{c,v}$ using Min-Sum across check neighbors.
- 3 Immediately update the variable posteriors in RAM:

$$L'_v = Q_{c,v} + R'_{c,v}$$

50% Memory Bandwidth Reduction

- By performing $L_v - R_{c,v}$ on the fly, the explicit 2D message matrix $R_{c,v}$ is **completely eliminated from memory!**
- Reduces BP memory payload by **50%**, doubling memory bus efficiency.

Optimization 3: Explicit AVX-512 SIMD Vectorization

Vector Register Mapping

16 parallel decoding shots (32-bit integer LLRs) map directly into one 512-bit AVX-512 register (`_mm512i`):

$$16 \times 32 \text{ bits} = 512 \text{ bits}$$

Core AVX-512 Intrinsic Operations

- `_mm512_abs_epi32`: Compute absolute LLR magnitudes for 16 shots simultaneously.
- `_mm512_min_epi32`: Track minimum LLRs (`min1`, `min2`) without conditional branching latency.
- `_mm512_sub_epi32` / `_mm512_add_epi32`: Vectorized posterior updates.
- `_mm512_mask_blend_epi32` / `_mm512_movepi32_mask`: Masked updates for active unconverged shots.

Optimization 4: Stochastic Check-Node Schedule Permutations

Breaking Trapping Sets with PRNG

- Fixed schedule $0 \rightarrow M - 1$ locks into symmetric LDPC trapping sets.
- **Solution:** Apply Fisher-Yates check-node permutation before each BP iteration using a fast **Xorshift64 PRNG**.
- Because memory is interleaved in 64-byte blocks, random check node hopping incurs **zero SIMD alignment penalty!**
- Resolves trapping sets in Color Codes and Bivariate Bicycle codes seamlessly.

Xorshift64 PRNG

```
1 inline uint64_t xorshift64(uint64_t &  
2     state) {  
3     uint64_t x = state;  
4     x ^= x << 13;  
5     x ^= x >> 7;  
6     x ^= x << 17;  
7     return state = x;  
}
```

Optimization 5: Ordered Statistic Decoding (OSD) Integration

Algebraic Fallback Pipeline

When BP fails to clear all syndrome bits after `max_iter`:

- ① **Reliability Ordering:** Sort variable nodes by absolute LLR magnitude $|L_v|$ (highest magnitude = highest reliability).
- ② **Information Set Basis:** Perform column-pivoted Gaussian elimination on H matrix over SIMD bitsets (`stim::simd_bits`).
- ③ **Perturbation Search:**
 - **OSD-0:** Hard decision on basis + solver solution.
 - **OSD-1:** Test 1-bit perturbations on least reliable columns.
 - **OSD-2:** Test 2-bit perturbations on candidate combinations.
- ④ Select candidate matching syndrome s with minimum total weight/cost.

Evaluation Setup

- **Hardware:** 48-vCPU Intel Xeon Workstation with AVX-512 vector support.
- **Sample Size:** 100,000 Monte Carlo shots per quantum code configuration.
- **Code Benchmark Families:**
 - Surface Codes ($d = 3, 5, 7, 9$, physical error rate $p = 0.001$).
 - Color Codes ($d = 5, 7$ Superdense).
 - Bivariate Bicycle Codes ($[[72, 12, 6]]$, $[[90, 8, 10]]$, $[[108, 8, 10]]$, $[[144, 12, 12]]$).

Empirical Benchmark Throughput Comparison

Quantum Code Family	Decoder Config	Baseline (shots/s)	Optimized (shots/s)	Speedup
Surface Code ($d = 3, p = 0.001$)	'serial-batched' (BP)	306,867.6	922,330.5	3.01x (+200.6%)
Surface Code ($d = 5, p = 0.001$)	'serial-batched' (BP)	30,728.0	161,361.1	5.25x (+425.1%)
Surface Code ($d = 7, p = 0.001$)	'serial-batched' (BP)	6,777.6	32,631.1	4.81x (+381.5%)
Surface Code ($d = 9, p = 0.001$)	'serial-batched' (BP)	2,258.9	8,660.9	3.83x (+283.4%)
Color Code Superdense ($d = 5$)	'serial-batched' (BP)	14,815.3	65,647.4	4.43x (+343.1%)
Color Code Superdense ($d = 7$)	'serial-batched' (BP)	3,062.9	9,399.2	3.07x (+206.9%)
Bivariate Bicycle $[[72, 12, 6]]$	'serial-batched' + OSD-0'	1,384.9	4,142.6	2.99x (+199.1%)
Bivariate Bicycle $[[72, 12, 6]]$	'serial-batched' + OSD-1'	978.5	2,165.6	2.21x (+121.3%)
Bivariate Bicycle $[[90, 8, 10]]$	'serial-batched' + OSD-0'	539.3	1,466.1	2.72x (+171.8%)
Bivariate Bicycle $[[108, 8, 10]]$	'serial-batched' + OSD-0'	414.3	1,096.1	2.65x (+164.6%)
Bivariate Bicycle $[[144, 12, 12]]$	'serial-batched' + OSD-0'	208.9	462.0	2.21x (+121.2%)

Performance Analysis & Speedup Drivers

Why Did Throughput Increase 2.2x to 5.25x?

- **L1 Cache locality:** 1D Interleaved layout eliminated L1 cache flushes during SIMD loads.
- **50% Bandwidth Reduction:** Eliminating $R_{c,v}$ 2D message tables doubled memory bus efficiency.
- **AVX-512 Intrinsics:** 16-shot SIMD vector registers executed without branch latency.

Trapping Set Resolution

- Stochastic check node permutations broke static trapping set loops in Bivariate Bicycle and Color codes.
- Average iterations to convergence dropped significantly across complex qLDPC topologies!

Next Steps for Tesseract-BP

- 1 **Vectorized OSD Gaussian Elimination:** Batching column-pivoted Gaussian elimination over 16-shot SIMD blocks when BP fails to converge.
- 2 **OSD-2 Priority Search Pruning:** Restricting perturbation search space candidates using LLR priority bounds.
- 3 **BP-Guided Search Integration:** Feeding BP posteriors directly into Tesseract A* Search heuristic cost matrices ($g(n) + h(n)$).

Key Takeaways

- **Tesseract Decoding Vision:** Tiered QEC workflow (BP $\rightarrow$ MultiPass $\rightarrow$ Full Tesseract Search).
- **High-Throughput BP Engine:** 1D Interleaved Flat Array + Horizontal Layered Serial Min-Sum.
- **AVX-512 Vectorization:** 16 parallel shot streams processed in a single 512-bit register.
- **LDPC Trapping Set Solution:** Stochastic Fisher-Yates schedule permutations.
- **Empirical Gains:** 2.2x to 5.25x throughput speedup across Surface, Color, and Bivariate Bicycle codes.

Questions & Discussion